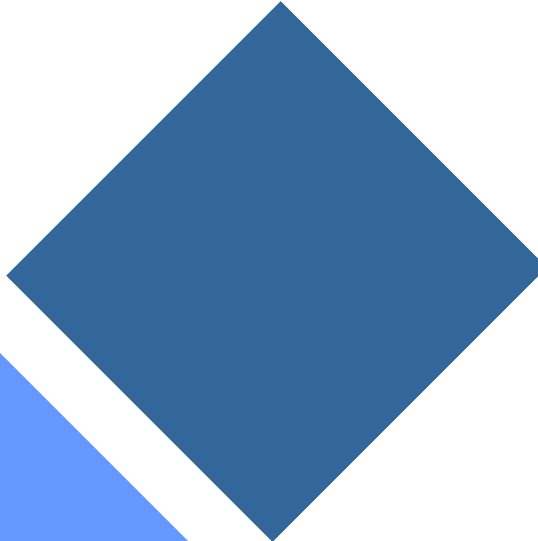


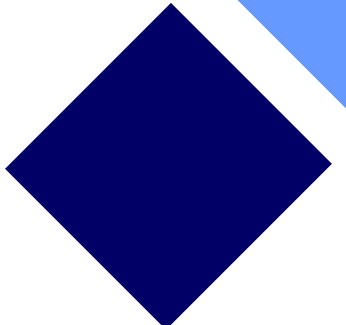
SP5: Optimización y Parquet

Isabel Pelaya Galindo Ibáñez
Esther Ibáñez Mingorance
José Luis López García
Juan Rafael Madolell Usero





Índice



- 1. Parquet vs CSV**
- 2. Snappy**
- 3. Predicado Pushdown**
- 4. Particionado**
- 5. Schema Merging**
- 6. Bucketing**
- 7. Z-Ordering**
- 8. Práctica**
- 9. Resultados**
- 10. Problemas**

1. PARQUET VS CSV

¿Qué es Parquet?: es un formato de almacenamiento columnar.

Parquet vs CSV: A diferencia de los archivos CSV (basados en filas), Parquet organiza datos por columnas, lo que permite una compresión superior, menor uso de disco y consultas analíticas mucho más rápidas al leer solo las columnas necesarias.

Gracias a esto:

- Se reduce significativamente el tiempo de lectura en grandes volúmenes de datos.
- Se optimiza el uso de memoria y almacenamiento.
- Se mejora el rendimiento en herramientas de análisis de datos.

2. SNAPPY

Snappy es un algoritmo de compresión usado para reducir el tamaño de los datos en Parquet.

Ventajas de Snappy:

- Compresión rápida.
- Descompresión aún más rápida.
- Buen equilibrio entre tamaño y rendimiento.

Resultado:

- Menor uso de disco.
- Lecturas más rápidas en Spark.

3. PREDICADO PUSHDOWN

Predicado Pushdown es una técnica que permite a Spark leer solo los datos necesarios desde el disco.

¿Cómo funciona?

- Aplica filtros directamente en el almacenamiento.
- Evita cargar datos innecesarios en memoria.

Ventaja clave:

- Mejora significativa del rendimiento en consultas.

4. PARTICIONADO

El particionado consiste en organizar los datos físicamente en carpetas según el valor de una columna clave.

¿Qué consigue?

- Spark lee solo las particiones necesarias.
- Evita escanear todo el dataset.

Ventajas:

- Reducción drástica del tiempo de lectura.
- Menor uso de recursos (CPU y memoria).
- Escalabilidad en grandes volúmenes de datos.

5. SCHEMA MERGING

Permite unificar automáticamente diferentes esquemas de datos dentro de un mismo dataset.

¿Cuándo se utiliza?

- Al añadir datos nuevos con columnas adicionales o cambios en la estructura.

¿Qué hace Parquet?

- Combina ambos esquemas sin perder información.

Ventajas:

- Evolución flexible del modelo de datos.
- No es necesario rehacer datasets antiguos.
- Facilita pipelines de datos dinámicos.

6. BUCKETING

El bucketing es una técnica que consiste en dividir los datos en un número fijo de grupos (buckets) según el valor de una columna.

Cómo funciona?

- Se aplica una función hash sobre una columna.
- Los datos se distribuyen en buckets predefinidos.

¿Para qué sirve?

- Optimizar operaciones costosas como:
 - joins

Ventajas:

- Reduce el movimiento de datos entre nodos.
- Mejora el rendimiento en consultas complejas.
- Especialmente útil en datasets muy grandes.

7. Z-ORDERING

Técnica que consiste en organizar los datos dentro de los archivos agrupando valores relacionados de varias columnas.

¿Cómo funciona?

- Convierte datos de múltiples columnas en un orden único.
- Coloca juntos en disco los valores que suelen consultarse en combinación.

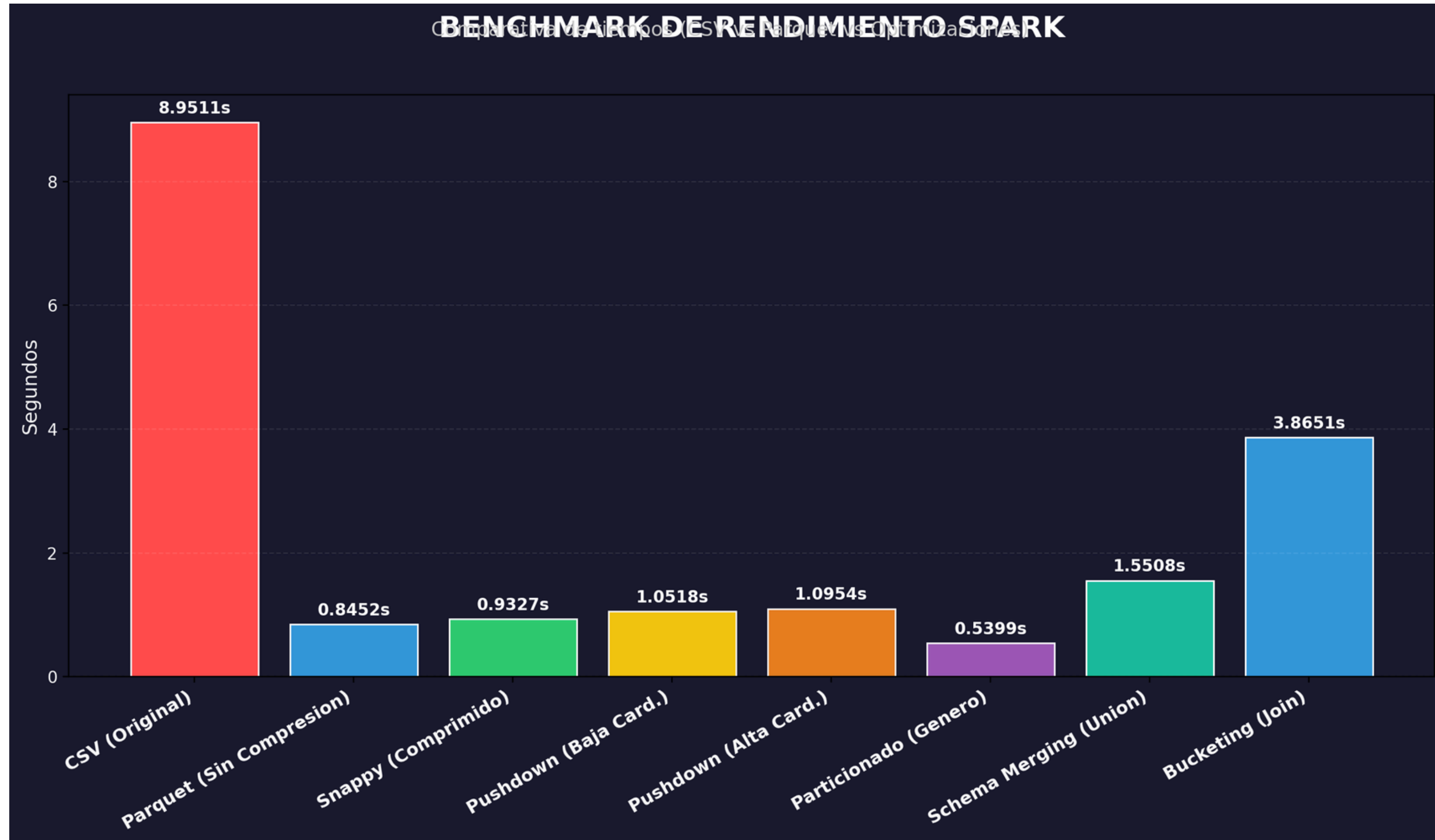
¿Qué consigue?

- Permite aplicar data skipping (evitar leer archivos innecesarios).
- Reduce la cantidad de datos leídos.

Ventajas:

- Mejora el rendimiento en consultas con múltiples filtros.
- Disminuye el tiempo de ejecución.
- Reduce el coste de lectura en grandes volúmenes de datos.

8. PRÁCTICA



9. RESULTADOS

- **CSV:** El CSV es un formato de texto plano que no está optimizado. Spark tiene que leer el archivo línea a línea y convertir cada registro de texto a objetos dentro de su memoria. Además, aunque solo queramos la suma de una columna, Spark tiene que cargar toda la fila completa, lo cual es ineficiente en disco.
- **Parquet (Sin Compresión):** Al pasar a Parquet, Spark usa un almacenamiento columnar. Esto significa que solo lee los valores de las columnas que pides en la consulta (gender y productivity_score), ignorando el resto. Solo con esto ya ahorramos muchísima lectura de disco.
- **Snappy (Comprimido):** Snappy reduce el tamaño del archivo en disco. Aunque Spark tiene que "descomprimirlo" al leerlo, como el archivo ocupa menos, tarda menos tiempo en viajar del disco duro a la memoria RAM de Spark, lo que suele ganar en velocidad total.

9. RESULTADOS

- **Pushdown (Baja Cardinalidad - Género):** Spark ya no lee todos los datos de la columna género. Parquet guarda metadatos sobre qué valores hay en cada bloque; Spark los mira y dice: "En este bloque no hay mujeres, lo salto". Se lee solo una parte de la información.
- **Pushdown (Alta Cardinalidad - IDs):** Es incluso un poco más alto que el de baja cardinalidad en ocasiones, pero permite buscar un registro concreto (como el estudiante 5000) casi al instante. Parquet conoce los rangos de IDs por bloque y puede ir directo al grano.
- **Particionado (Género):** Spark ve que los datos de mujeres están en la carpeta gender=Female. En lugar de buscar dentro de un archivo grande, ignora todas las demás carpetas y entra directo a la de mujeres. Es la optimización de lectura de disco más potente.

9. RESULTADOS

- **Schema Merging:** Al leer varios archivos con esquemas distintos para unirlos, Spark tiene que leer el encabezado (metadato) de cada uno para construir el esquema resultante "al vuelo". Ese proceso de comparar y unificar los metadatos es lo que añade ese tiempo extra comparado con un Parquet normal.
- **Bucketing:** Es la barra más alta después del CSV porque estamos haciendo un JOIN (cruzar la tabla consigo misma). El JOIN es la operación más costosa en Big Data. Sin embargo, al usar Bucketing, el tiempo es mucho menor que si lo hiciéramos sin él, porque Spark ya tiene los datos agrupados por `student_id` en disco y se ahorra la operación de "Shuffle" (mover datos de un lado a otro).

10. PROBLEMAS

Problema:

El Parquet particionado tardaba lo mismo o más que el Parquet normal debido a que la consulta no incluía un filtro por la columna de partición (gender), impidiendo a Spark aplicar partition pruning y obligándole a leer todos los datos.

Solución:

Se descartó esa columna y se utilizó una de baja cardinalidad (como gender), adecuada para particionado.

10. PROBLEMAS

Problema:

El programa se bloqueaba al particionar porque se utilizó la columna `student_id`, con alta cardinalidad (20.000 valores únicos), generando miles de carpetas y saturando el sistema de archivos.

Solución:

Se modificó la consulta del benchmark para incluir un filtro por género, demostrando así la verdadera potencia del particionado al saltarse la lectura de datos innecesarios.

**Muchas
gracias**

